# 武汉大学计算机学院
# 本科生课程设计报告

## 选题与团队组建报告

专业: 计算级弘毅班

课程: 计算机综合项目实践

指导教师: 谭小琼

团队名称: token

团队成员: 周炜、黎宇恒

学年学期: 2026-2027 学年第二学期

武汉大学计算机学院
2026 年 3 月 13 日

# 目录

1 课题名称 ..... 3
2 选题目的和意义 ..... 3
2.1 选题目的 ..... 3
2.2 选题意义 ..... 3
3 研究内容、研究方法、技术路线及可行性分析 ..... 3
3.1 研究内容 ..... 3
3.2 研究方法 ..... 4
3.3 技术路线 ..... 4
3.4 可行性分析 ..... 5
4 项目特色与创新点 ..... 5
5 进度安排 ..... 6
6 成员名单、各成员角色及任务分配 ..... 7
6.1 团队协作规范 ..... 7

# 1 课题名称

**基于大语言模型的个人健康智能分析与管理系统（产品名：HealthMate 健康伴侣）**

拟采用开发技术：前端（React / React Native）+ 后端（Python/FastAPI）+ 数据库（MySQL+Redis）+ AI 大模型 API 接入 + 容器化部署（Docker）

# 2 选题目的和意义

## 2.1 选题目的

当下亚健康状态愈发普遍，传统健康管理工具仅停留在数据记录层面，缺乏个性化分析与指导。本项目开发个人健康智能助手，通过记录用户睡眠、饮食、运动等数据，结合 AI 技术生成个性化健康建议与干预计划，同时让团队成员掌握软件工程全流程开发方法，实践前后端分离、AI 模型集成等前沿技术，培养 Scrum 敏捷开发与团队协作能力。

## 2.2 选题意义

本选题契合“人工智能、云计算与大数据”技术发展趋势，响应“健康中国 2030”战略，探索 AI 在健康管理领域的实际应用。项目开发能让团队积累企业级项目开发经验，提升解决实际工程问题的能力，产出的可运行健康助手原型具备一定社会应用价值，同时高度匹配课程对工程实践、完整软件生命周期开发的核心要求。

# 3 研究内容、研究方法、技术路线及可行性分析

## 3.1 研究内容

核心开发 Web 端+移动端适配的健康管理应用，包含五大功能模块：

- 健康数据采集模块：支持手动录入、自然语言/语音录入睡眠、饮食、运动等结构化数据，可选图片识别食物自动提取热量；
- AI 智能分析模块：汇总用户当日及历史数据，调用大模型生成个性化健康评估与次日行动建议；

- 数据可视化模块：以折线图、雷达图等形式展示周/月健康数据趋势，对比历史均值；
- 目标管理与打卡模块：支持用户设定健康目标，AI 动态调整建议，将建议转化为待办任务，用户可打卡完成形成闭环；
- 匿名社交对比模块（可选）：让用户匿名对比同龄人群健康趋势，增强参与感。

## 3.2 研究方法

- 文献调研：查阅 AI 在健康管理领域的应用文献，借鉴主流技术方案；
- Scrum 敏捷开发：将开发周期划分为多个 Sprint 迭代，分阶段推进开发、测试与优化；
- 前后端分离开发：前端负责界面交互，后端提供 RESTful API，AI 服务作为独立模块集成；
- 测试驱动开发：编写单元测试、集成测试与功能测试，保障系统质量；
- Git 版本控制：遵循 Git Flow 规范，分仓开发、规范提交，记录代码与文档贡献。

## 3.3 技术路线

表 1 技术路线表

| 技术层级 | 核心选型 |
| --- | --- |
| 前端 | React/React Native + ECharts（可视化）+ Ant Design Mobile/Element Plus（UI）+ Axios（网络请求） |
| 后端 | Python+FastAPI + MySQL（结构化数据）+ Redis（缓存/队列） |
| AI 服务 | LangChain（编排）+ 国内大模型 API（DeepSeek/Kimi/智谱 GLM） |
| 部署运维 | Docker + Nginx + 阿里云/腾讯云学生服务器 |
| 项目管理 | Scrum 敏捷框架 + Git/GitHub/GitLab（版本控制） |

## 3.4 可行性分析

- 技术可行性：前后端技术栈成熟，AI 大模型 API 接入简单、文档完善，数据可视化、容器化部署均有大量开源案例参考；
- 团队可行性：2 人团队技能互补，分别专精前后端，可兼任 Scrum 全角色，覆盖开发、测试、运维全流程；
- 时间可行性：课程共 14 周，按敏捷迭代分阶段开发，核心功能聚焦数据流转与 AI 分析，工作量适中，预留测试与优化时间；
- 经济可行性：使用免费开发工具，云服务器、AI API 可享受学生优惠/低成本套餐，可控开发成本。

# 4 项目特色与创新点

1. **个性化连续式 AI 建议**：轻量化应用 RAG 检索增强生成技术，向大模型请求时携带用户近 7 天健康数据作为上下文，让建议具备连续性、针对性，避免泛化模板化内容；
2. **健康管理闭环设计**：将 AI 文本建议解析为结构化的待办打卡任务，形成“数据记录-AI 分析-生成建议-行动打卡-数据反馈”的完整闭环，提升实用性与用户黏性；
3. **多模态极简数据录入**：支持自然语言、语音甚至图片识别的多模态数据录入方式，自动提取结构化信息存入数据库，大幅降低用户输入门槛；
4. **动态化目标管理**：根据用户健康目标的完成情况，AI 实时动态调整建议与任务，贴合用户实际健康状态，体现智能适配特性；
5. **轻量化易扩展架构**：采用前后端分离+微服务思路设计，未来可无缝接入可穿戴设备数据、增加更多健康指标，同时兼顾隐私保护，用户数据本地化存储，AI 调用仅传递特征数据；
6. **直观化数据可视化**：打造个人健康仪表盘，多维度展示健康数据趋势与对比结果，让用户清晰感知自身健康状态变化。

# 5 进度安排

本项目严格遵循课程 14 周时间线推进，分阶段完成开发并提交指定交付物，具体安排如下：

表 2 进度安排表

| 周次 | 主要任务 | 核心交付物 |
| --- | --- | --- |
| 第 1-2 周 | 项目调研、选题确认、团队组建与角色划分，注册云服务器与 AI API 账号，环境搭建 | 选题及团队组建报告 |
| 第 2-3 周 | 需求分析、绘制业务流程图/用例图，设计接口规范，完成 UI 原型草图设计 | 需求规格说明书 |
| 第 3-4 周 | 系统架构设计、MySQL 数据库 E-R 图与表结构设计，搭建前后端开发脚手架 | 系统设计说明书 |
| 第 4-6 周 | Sprint1：完成用户注册登录、基础健康数据表单的增删改查及数据库存储 | 功能代码、Git 提交记录 |
| 第 7-9 周 | Sprint2：核心攻坚，后端完成 AI 大模型接入与 Prompt 工程，前端实现 AI 建议渲染与数据可视化 | 功能代码、Git 提交记录 |
| 第 10-12 周 | Sprint3：完成目标管理与打卡模块，系统全链路联调、Bug 修复与代码重构 | 功能代码、阶段工作小结 |
| 第 9-11 周 | 系统集成测试、功能优化，Docker 容器化打包并部署至云服务器 | 可运行系统、测试报告 |
| 第 12-14 周 | 答辩准备、系统演示视频制作，撰写项目实践报告，整理项目所有资料 | 答辩 PPT、项目实践报告 |

# 6 成员名单、各成员角色及任务分配

团队共 2 名成员，均为计算机科学与技术专业，在 Scrum 敏捷框架下兼任多角色，覆盖开发、管理、测试、运维全流程，分工明确、技能互补，具体分配如下：

**表 3 任务分配表**

| 姓名 | 学号 | Scrum 角色 | 具体任务分配 |
|---|---|---|---|
| 黎宇恒（前端） | 2023302112014 | Scrum Master+前端开发+测试+运维 | 1. 管理：把控整体开发进度，组织每周迭代沟通，推进 Scrum 流程落地；<br>2. 开发：负责前端界面 UI/UX 设计、组件化开发，ECharts 数据可视化实现，与后端 API 对接联调，基于 React 完成用户登录、任务打卡等交互功能；<br>3. 测试：共同负责系统测试工作，包括前端测试用例编写、跨端兼容性测试、用户端功能测试，协同完成测试报告；<br>4. 运维：配合后端完成 Docker 容器化部署、云服务器环境搭建与维护，协助完成系统上线相关工作。 |
| 周炜（后端） | 2023302112015 | Product Owner+后端开发+测试+运维 | 1. 管理：担任产品负责人，梳理业务流与数据字典，管理产品 Backlog，对接指导教师确认需求与开发方向；<br>2. 开发：负责 MySQL/Redis 数据库设计与维护，FastAPI 后端 API 开发，核心完成 AI 大模型接入、Prompt 工程优化与多模态数据解析；<br>3. 测试：完成系统测试，包括后端接口测试、集成测试，协同编写测试用例与测试报告。<br>4. 运维：共同负责运维工作，主导 Docker 容器化部署、云服务器环境搭建与维护，协同完成系统上线； |

**说明：由于团队仅有两人，角色适当合并，但保证各角色职责覆盖。两人将通过 Git 进行代码协作，确保提交记录清晰可追溯。**

## 6.1 团队协作规范

1. 严格遵循 Git Flow 规范，接口文档通过 Swagger 实时同步；
2. 所有代码、文档提交需标注清晰的修改说明，按 **feat/fix/docs/style** 等规范提交，确保 Git 贡献记录可追溯；

3. 每周开展迭代复盘，同步开发进度、解决协作问题，确保项目按计划推进。